

```
//
=====

=====
// Datei: html_data.cpp
// Projekt: ESP32-CAM-BOT-MODULE
// Beschreibung: Enthält ausschließlich das HTML/JS/CSS-Webinterface.
// Gespeichert im Flash-Speicher (PROGMEM) zur RAM-Schonung.
//
=====

=====
#include <Arduino.h>

// "extern "C"" verhindert, dass der C++ Compiler den Variablennamen für den Linker
verändert.
// "PROGMEM" sorgt dafür, dass dieser riesige Text im Flash-Speicher bleibt und nicht das
knappe RAM blockiert.
// "R"rawliteral(...)" ist ein Raw-String-Literal. Alles dazwischen wird exakt so
übernommen, inklusive Zeilenumbrüchen.
extern "C" const char INDEX_HTML[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html lang="de">
<head>
  <!-- Definiert den Zeichensatz (UTF-8 unterstützt Umlaute und Emojis) -->
  <meta charset="UTF-8">
  <!-- Verhindert, dass der Nutzer auf dem Smartphone die Seite versehentlich zoomed
(wichtig für Joystick-Bedienung) -->
  <meta name="viewport" content="width=device-width, initial-scale=1, maximum-
scale=1, user-scalable=no">
  <title>CamBot Joystick & Tuning</title>

  <style>
/* --- CSS DESIGN & STYLING --- */

/* :root definiert globale Variablen, um Farben zentral zu steuern */
```

```

:root {
  --bg: #0d0d0d; /* Hintergrundfarbe (fast Schwarz) */
  --panel: #1a1a1a; /* Panel-Hintergrund (Dunkelgrau) */
  --border: #2e2e2e; /* Rahmenfarbe */
  --accent: #e8c547; /* Akzentfarbe (Gelb/Gold) */
  --text: #e0e0e0; /* Textfarbe (Hellgrau) */
  --muted: #666; /* Ausgegrauter Text für Statusmeldungen */
}

/* Universeller Selektor (*): Setzt Abstände für alle Elemente auf 0 */
* { box-sizing: border-box; margin: 0; padding: 0; }

/* Stylt die gesamte Webseite */
body {
  background: var(--bg); color: var(--text); font-family: 'Courier New', monospace;
  display: flex; flex-direction: column; align-items: center; min-height: 100vh;
  padding: 12px; gap: 12px;
  user-select: none; -webkit-user-select: none; /* Verhindert Textmarkierungen beim
Tippen */
}

/* Überschrift */
h1 { font-size: 1.1rem; letter-spacing: 0.25em; color: var(--accent); text-transform:
uppercase; padding: 6px 0; }

/* Die Box, in der das Kamerabild angezeigt wird */
#stream-box {
  width: 100%; max-width: 640px; background: #000; border: 1px solid var(--border);
  border-radius: 4px; overflow: hidden; aspect-ratio: 4/3; display: flex; align-items: center;
justify-content: center;
}

/* Das eigentliche Bild (wird per Standard ausgeblendet, bis der Stream startet) */
#stream { width: 100%; height: 100%; object-fit: contain; display: none; }

```

```

/* Platzhaltertext, wenn kein Video läuft */
#stream-placeholder { color: var(--muted); font-size: 0.8rem; letter-spacing: 0.1em; }

/* Reihe für die Buttons unter dem Video */
.btn-row { display: flex; gap: 10px; width: 100%; max-width: 640px; }

/* Allgemeines Design für alle Buttons */
button {
  flex: 1; padding: 10px; border: 1px solid var(--border); border-radius: 3px;
  background: var(--panel); color: var(--text); font-family: inherit; font-size: 0.85rem;
  letter-spacing: 0.1em; transition: border-color 0.15s, color 0.15s; cursor: pointer;
}
/* Hover-Effekt (Mauszeiger drüber) */
button:hover { border-color: var(--accent); color: var(--accent); }
/* Active-Effekt (Button wird gedrückt) */
button:active { background: #222; }

/* Spezialklasse für den Licht-Button, wenn das Licht eingeschaltet ist */
#btn-licht.an { border-color: var(--accent); color: var(--accent); }

/* Container für den Joystick */
.controls {
  width: 100%; max-width: 640px; background: var(--panel); border-radius: 4px;
  border: 1px solid var(--border); padding: 20px; display: flex; justify-content: center;
  align-items: center;
}

/* Der äußere, gestrichelte Kreis des Joysticks */
#joystick-zone {
  width: 180px; height: 180px; background: #111; border: 2px dashed var(--border);
  border-radius: 50%; position: relative; touch-action: none; /* touch-action: none
verhindert Scrollen beim Ziehen */
}

```

```

/* Der bewegliche Knopfteil (Knob) in der Mitte des Joysticks */
#joystick-knob {
  width: 54px; height: 54px; background: var(--panel); border: 2px solid var(--accent);
  border-radius: 50%; position: absolute; left: 61px; top: 61px; cursor: pointer;
}

/* Textbereich für Status- und Fehlermeldungen ganz unten */
#status { font-size: 0.7rem; color: var(--muted); letter-spacing: 0.08em; text-align:
center; }

/* Das Kalibrierungs-Panel (ausklappbares Tuning-Menü) */
.tuning-panel {
  width: 100%; max-width: 640px; background: var(--panel); border: 1px solid var(--
border);
  border-radius: 4px; padding: 12px; display: flex; flex-direction: column; gap: 10px;
}

/* Der anklickbare Titel des Tuning-Panels */
.tuning-title {
  font-size: 0.8rem; color: var(--accent); cursor: pointer; text-align: center;
  letter-spacing: 0.1em; padding: 4px; border-bottom: 1px solid #222;
}

/* Inhalt des Tuning-Panels (standardmäßig unsichtbar: display: none) */
.tuning-content { display: none; flex-direction: column; gap: 8px; }
/* Wird per JS hinzugefügt, um das Menü sichtbar zu machen */
.tuning-content.open { display: flex; }

/* Zeile für Beschriftung + Eingabefeld im Menü */
.input-group { display: flex; justify-content: space-between; align-items: center; font-
size: 0.8rem; }
/* Design der Nummerneingabefelder */
.input-group input {

```

```

background: #111; border: 1px solid var(--border); color: #fff;
padding: 6px; width: 80px; text-align: center; font-family: inherit; border-radius: 3px;
}
/* Effekt, wenn man in ein Eingabefeld hineinklickt */
.input-group input:focus { border-color: var(--accent); outline: none; }
/* Der "Werte anwenden"-Button */
#btn-save { background: #222; margin-top: 5px; max-height: 35px; }
</style>
</head>
<body>

```

```

<!-- --- HTML STRUKTUR (WAS MAN SIEHT) --- -->

```

```

<h1>&#9654; CamBot</h1> <!-- Zeigt ein Play-Symbol und den Namen an -->

```

```

<!-- Video-Stream Box -->

```

```

<div id="stream-box">

```

```

  <img id="stream" alt="stream">

```

```

  <span id="stream-placeholder">STREAM GESTOPPT</span>

```

```

</div>

```

```

<!-- Buttons für Stream und LED-Licht -->

```

```

<div class="btn-row">

```

```

  <button id="btn-stream" onclick="toggleStream()">&#9654; STREAM</button>

```

```

  <button id="btn-licht" onclick="toggleLicht()">&#9788; LICHT</button>

```

```

</div>

```

```

<!-- Der Joystick-Bereich -->

```

```

<div class="controls">

```

```

  <div id="joystick-zone">

```

```

    <div id="joystick-knob"></div>

```

```

  </div>

```

```

</div>

```

```

<!-- Tuning / Kalibrierungs-Bereich -->

```

```

<div class="tuning-panel">
  <div class="tuning-title" onclick="toggleTuning()">&#9881; KALIBRIERUNG
(KLICKEN)</div>
  <div id="tuning-box" class="tuning-content">
    <div class="input-group">
      <label>Mindest-PWM (0-255):</label>
      <input type="number" id="inp-min" value="70" min="0" max="255">
    </div>
    <div class="input-group">
      <label>Max Speed (0.0 - 1.0):</label>
      <input type="number" id="inp-max" value="0.85" step="0.05" min="0" max="1">
    </div>
    <div class="input-group">
      <label>Korrektur Links (0.5 - 1.0):</label>
      <input type="number" id="inp-kl" value="1.00" step="0.01" min="0.5" max="1">
    </div>
    <div class="input-group">
      <label>Korrektur Rechts (0.5 - 1.0):</label>
      <input type="number" id="inp-kr" value="1.00" step="0.01" min="0.5" max="1">
    </div>
    <button id="btn-save" onclick="sendCalibration()">WERTE ANWENDEN</button>
  </div>
</div>

```

```

<!-- Status-Textzeile ganz unten -->

```

```

<div id="status"></div>

```

```

<!-- --- JAVASCRIPT LOGIK (FUNKTIONEN) --- -->

```

```

<script>

```

```

// Globale Statusvariablen

```

```

let streamAktiv = false;

```

```

let lichtAn = false;

```

```

let sendTimer = null; // Timer, um die Befehlsübertragung zu drosseln (Debounce)

```

```

let streamUrl = null; // Speichert die IP/URL des Kamera-Streams

```

```

// DOM-Elemente referenzieren (JavaScript Zugriff auf HTML erlauben)
const zone = document.getElementById('joystick-zone');
const knob = document.getElementById('joystick-knob');

// Konstanten für die Joystick-Berechnung
const maxDistance = 60; // Maximaler Radius, den der Knopf gezogen werden darf (in
Pixeln)
const startX = 61; // Startposition X des Knopfs im Kreis (zentriert)
const startY = 61; // Startposition Y des Knopfs im Kreis (zentriert)
let active = false; // Merkt sich, ob der Joystick gerade gedrückt/gezogen wird

// Holt beim Laden der Seite die Stream-URL vom ESP32-Server ab (z.B. die IP-Adresse
mit Port 81)
fetch('/streamurl').then(r => r.text()).then(u => { streamUrl = u; });

// Event-Listener für Touch-Eingaben (Smartphones)
zone.addEventListener('touchstart', startMove, {passive: false});
window.addEventListener('touchend', stopMove);
window.addEventListener('touchmove', move, {passive: false});

// Event-Listener für Maus-Eingaben (PC)
zone.addEventListener('mousedown', startMove);
window.addEventListener('mouseup', stopMove);
window.addEventListener('mousemove', move);

// Startet die Bewegung, sobald der Nutzer auf den Joystick klickt/tippt
function startMove(e) { active = true; move(e); }

// Setzt den Joystick zurück, wenn der Nutzer loslässt
function stopMove() {
  if (!active) return;
  active = false;
}

```

```
knob.style.left = startX + 'px'; // Springt zurück in die Mitte (X)
```

```
knob.style.top = startY + 'px'; // Springt zurück in die Mitte (Y)  
update(0, 0); // Sendet "Stopp" (Fahrt=0, Kurve=0) an den Bot  
}
```

```
// Berechnet die Position des Joysticks während des Ziehens
```

```
function move(e) {  
  if (!active) return;  
  e.preventDefault(); // Verhindert das Scrollen der Webseite beim Ziehen
```

```
  let clientX, clientY;
```

```
  // Unterscheidung zwischen Touch- und Mauskoordinaten
```

```
  if (e.touches) { clientX = e.touches(0).clientX; clientY = e.touches(0).clientY; }  
  else { clientX = e.clientX; clientY = e.clientY; }
```

```
  // Holt die Position des äußeren Joystick-Kreises auf dem Bildschirm
```

```
  const rect = zone.getBoundingClientRect();  
  const centerX = rect.left + rect.width / 2;  
  const centerY = rect.top + rect.height / 2;
```

```
  // Abstand von der Mitte berechnen
```

```
  let deltaX = clientX - centerX;  
  let deltaY = clientY - centerY;
```

```
  // Satz des Pythagoras, um die tatsächliche Luftlinie/Distanz zu berechnen
```

```
  let distance = Math.sqrt(deltaX * deltaX + deltaY * deltaY);
```

```
  // Wenn der Finger weiter weg ist als erlaubt, begrenze den Knopf auf den Maximalradius  
  (maxDistance)
```

```
  if (distance > maxDistance) {  
    deltaX = (deltaX / distance) * maxDistance;
```



```
deltaY = (deltaY / distance) * maxDistance;  
}
```

```
// Verschiebt den HTML-Knopf visuell auf dem Bildschirm
```

```
knob.style.left = (startX + deltaX) + 'px';
```

```
knob.style.top = (startY + deltaY) + 'px';
```

```
// Berechnet die Werte für den Bot (-100 bis +100 Prozent)
```

```
// deltaY ist negativ, wenn man nach oben zieht. Ein Minus davor macht es positiv für  
"Vorwärtsfahrt".
```

```
let fahrt = Math.round((-deltaY / maxDistance) * 100);
```

```
let kurve = Math.round((deltaX / maxDistance) * 100);
```

```
update(fahrt, kurve);
```

```
}
```

```
// Verhindert, dass bei jeder minimalen Fingerbewegung sofort Daten gesendet werden  
(Drosselung auf max. alle 40ms)
```

```
function update(f, k) {
```

```
  clearTimeout(sendTimer);
```

```
  if (f === 0 && k === 0) {
```

```
    sendCtrl(0, 0); // Stopp-Befehl wird sofort ohne Verzögerung gesendet
```

```
  } else {
```

```
    sendTimer = setTimeout(() => sendCtrl(f, k), 40); // Wartet 40 Millisekunden ab
```

```
  }
```

```
}
```

```
// Sendet die Fahrdaten über ein HTTP-Get-Request an den ESP32-Webserver (/ctrl?
```

```
fahrt=X&kurve=Y)
```

```
function sendCtrl(f, k) {
```

```
  fetch('/ctrl?fahrt=' + f + '&kurve=' + k)
```

```
  .then(() => setStatus('fahrt=' + f + ' kurve=' + k)) // Zeigt aktuelle Werte unten im Status an
```

```
  .catch(() => setStatus('FEHLER - Verbindung verloren')); // Fehler, wenn der ESP32 nicht
```

antwortet

```
}
```

```
// Blendet das Tuning-Menü ein oder aus (schaltet die CSS-Klasse "open" um)
```

```
function toggleTuning() {
```

```
document.getElementById('tuning-box').classList.toggle('open');
```

```
}
```

```
// Liest die Tuning-Eingabefelder aus und schickt sie an die URL "/cal" des ESP32
```

```
function sendCalibration() {
```

```
const minPwm = document.getElementById('inp-min').value;
```

```
const maxSpeed = document.getElementById('inp-max').value;
```

```
const korrL = document.getElementById('inp-kl').value;
```

```
const korrR = document.getElementById('inp-kr').value;
```

```
fetch(/cal?min_pwm=${minPwm}&max_speed=${maxSpeed}&korr_l=${korrL}&korr_r=${korrR})
```

```
.then(() => setStatus('Kalibrierung erfolgreich angewendet!'))
```

```
.catch(() => setStatus('FEHLER beim Senden der Kalibrierung'));
```

```
}
```

```
// Schaltet den Video-Stream an oder aus
```

```
function toggleStream() {
```

```
const img = document.getElementById('stream');
```

```
const ph = document.getElementById('stream-placeholder');
```

```
const btn = document.getElementById('btn-stream');
```

```
streamAktiv = !streamAktiv; // Zustand umkehren
```

```
if (streamAktiv && streamUrl) {
```

```
img.src = streamUrl; // Weist dem Bild die Stream-URL zu (Video startet)
```

```
img.style.display = 'block'; // Zeigt das Bild an
```

```
ph.style.display = 'none'; // Versteckt den Platzhaltertext
```

```
btn.textContent = '\u23F9 STREAM'; // Ändert den Button-Text zu einem Stopp-Symbol +  
STREAM
```

```

setStatus('Stream laeuft auf Port 81');
} else {
img.src = ''; // Leert die Quelle (stoppt den Datenstrom im Hintergrund)
img.style.display = 'none'; // Versteckt das Bild
ph.style.display = 'block'; // Zeigt den Platzhaltertext "STREAM GESTOPPT"
btn.textContent = '\u25B6 STREAM'; // Ändert den Button-Text wieder zum Play-Symbol
streamAktiv = false;
setStatus('Stream gestoppt');
}
}

// Schaltet die LED am Bot ein oder aus
function toggleLicht() {
fetch('/licht').then(r => r.text()).then(t => {
// Der ESP32 antwortet entweder mit dem Text "AN" oder "AUS"
lichtAn = (t === 'AN');
// Fügt die CSS-Klasse "an" hinzu oder entfernt sie, um den Button gelb leuchten zu lassen
document.getElementById('btn-licht').classList.toggle('an', lichtAn);
setStatus('Licht ' + t);
}).catch(() => setStatus('FEHLER - Licht'));
}

// Hilfsfunktion, um schnell Text in die Statuszeile ganz unten zu schreiben
function setStatus(msg) { document.getElementById('status').textContent = msg; }
)rawliteral";

```